CAHIER DE RÉALISATION

Orchestration de ressources multi-VM et multi-réseaux avec Heat et Auto-scaling

UNIVERSITÉ SAINT JEAN

Phase 4 — Implémentation & Livraison de la plateforme

Rédigé par : Ngouo Franck Leonel • Moudio Abega Laurent Stéphane

Supervisé par : M. NGUIMBUS Emmanuel

Master 2 SSI — Groupe 1 — Mars 2026

PLAN DE PRÉSENTATION

Sommaire

1 Introduction & Rappel des objectifs

2 Architecture technique — Modèle MVT + Service Layer

3 Composants réalisés — Backend (5 services)

4 Composants réalisés — Frontend (5 pages)

5 Déploiement Heat — Templates HOT et composants réutilisables

6 Monitoring temps réel & Auto-scaling

7 Bilan des cas d'utilisation

8 Conclusion & Perspectives

1 — INTRODUCTION

Introduction & Rappel des objectifs

Le présent cahier de réalisation expose l'implémentation concrète de la plateforme Heat Orchestration Platform — une application web full-stack permettant la gestion d'infrastructures cloud OpenStack via Heat, avec monitoring temps réel et auto-scaling intelligent. La solution répond aux quatre besoins fonctionnels (BF1-BF4) identifiés lors de l'analyse.

BF1

Déploiement automatisé

Interface web pour créer, gérer et supprimer des stacks Heat depuis un template unique.

BF2

Scaling automatique

Politique configurable avec seuils CPU/RAM, cooldown et historique complet des actions.

BF3

Modularité des templates

Bibliothèque de composants réutilisables, éditeur YAML intégré et validation syntaxique.

BF4

Intégration monitoring

Agent Python déployé dans les VMs, transmission HTTP et WebSocket pour métriques temps réel.

2 — ARCHITECTURE

Architecture technique — Modèle MVT + Service Layer

VIEW	**frontend/templates/ + frontend/static/** HTML Jinja2 • CSS • JavaScript Vanilla • Chart.js
TEMPLATE (Routes)	**backend/routes/** Contrôleurs Flask — reçoit requêtes HTTP, valide les entrées, appelle les services, retourne JSON
SERVICE Layer	**backend/services/** Logique métier — OpenStack, auto-scaling, métriques, résolution UUID
MODEL	**backend/models/** SQLAlchemy ORM — Stack, VM, Metric, ScalingPolicy, Template • SQLite (dev) → PostgreSQL (prod)

Pile technologique

- Python 3.10 + Flask 3.1
- SQLAlchemy ORM (SQLite / PostgreSQL)
- OpenStackSDK (Heat, Nova, Neutron, Keystone)
- Flask-SocketIO (WebSocket)
- JavaScript Vanilla + Chart.js
- Jinja2 (templating HTML)
- psutil (agent métriques VMs)
- YAML / HOT (templates Heat)

3 — BACKEND

Composants réalisés — Backend (Services)

1	**openstack_service.py**	Fabrique de clients OpenStack (Heat, Nova, Neutron) avec authentification Keystone automatique. Centralise toutes les connexions à l'infrastructure cloud.
2	**stack_service.py** CU2.1–2.5	Gestion du cycle de vie complet des stacks Heat : create, list, update, delete et validate template. Couvre les cas d'utilisation CU2.1 à CU2.5.
3	**vm_service.py**	Gestion Nova : liste, détails, resize asynchrone via thread de fond, start et stop des instances. Résolution des flavor ID vers leurs noms lisibles.
4	**metrics_service.py** CU4.1–4.2	Réception des métriques des agents, résolution hostname→UUID Nova, stockage en base de données et émission WebSocket vers les clients connectés.
5	**scaling_service.py** CU3.1–3.3	Moteur d'auto-scaling : check_and_execute() déclenché à chaque POST de métrique, vérification du cooldown et enregistrement de l'historique (ScalingHistory).

5 services Python	15 routes API REST	1 serveur WebSocket	Threads async Nova

4 — FRONTEND

Composants réalisés — Frontend (5 pages)

1 Dashboard

index.html + dashboard.js

Compteurs globaux (stacks, VMs, métriques), état des services OpenStack et liste d'activité récente.

2 Stacks

stacks.html + stacks.js

Tableau des stacks Heat, formulaire de création, vue détaillée des ressources, outputs, événements, update et suppression.

3 VMs

vms.html + vms.js

Liste des instances Nova avec panneau slide-in : informations détaillées, configuration du scaling, resize, start et stop.

4 Monitoring

monitoring.html + monitoring.js

4 graphiques Chart.js rafraîchis en temps réel via WebSocket. Configuration de la politique d'auto-scaling (seuils, cooldown).

5 Templates

templates_page.html + templates.js

Bibliothèque de composants réutilisables, éditeur YAML avec coloration syntaxique, validation Heat et squelettes de templates.

5 — HEAT / TEMPLATES HOT

Déploiement Heat — Templates HOT & Composants réutilisables

Templates intégrés (builtin)

main_stack.yaml

VM + Security Group + IP flottante + Agent de métriques

vm_template.yaml

VM simple sans IP flottante — template de base minimal

security_group.yaml

Security Group Heat dédié aux stacks déployés

network_template.yaml

Réseau (usage limité par flat network DevStack)

Composants réutilisables (CU1.2 / CU1.3)

vm_simple.yaml

VM minimale — 6 paramètres injectables

vm_with_agent.yaml

VM + agent métriques injecté via user_data

security_group.yaml

SG avec règles SSH / HTTP / HTTPS / ICMP

6 — MONITORING & AUTO-SCALING

Monitoring temps réel & Auto-scaling

Agent VM (psutil)	metrics_ service.py	Base de données	WebSocket (SocketIO)	Chart.js (dashboard)
POST toutes les 30 s	Résolution hostname→UUID	SQLite / ORM Stockage	Émission room UUID	Graphiques temps réel

Moteur d'auto-scaling — ScalingService (CU3.1 / CU3.2 / CU3.3)

1 Réception de la métrique

À chaque POST /api/metrics/alert, ScalingService.check_and_execute() est appelé immédiatement.

2 Comparaison aux seuils

cpu_percent > threshold_up (défaut 80%) → Scale UP | cpu_percent < threshold_down (défaut 20%) → Scale DOWN

3 Vérification du cooldown

Si la dernière action est intervenue il y a moins de N secondes (ex : 60 s), l'opération est ignorée et journalisée.

4 Resize asynchrone Nova

VMService.resize_vm() lance un thread de fond → retour HTTP immédiat, sans timeout côté interface.

5 Confirmation Nova

_thread_resize() interroge Nova toutes les 10 s et confirme automatiquement quand l'état passe à VERIFY_RESIZE.

6 Enregistrement historique

ScalingHistory stocke : direction, flavor_avant, flavor_après, valeur_métrique, timestamp — consultable via CU4.3.

7 — BILAN DES CAS D'UTILISATION

Bilan des cas d'utilisation

9	4	1	14
CU réalisés	CU partiels	CU non réalisés	CU au total

Cas d'utilisation	Description	Statut
CU1.1 → 1.4	Gestion des templates modulaires, validation et composants réutilisables	✓✓ Réalisé
CU1.5	Bibliothèque de templates avec catégories et système de tags	⚡ Partiel
CU2.1 / 2.3 / 2.5	Déploiement simple, consultation de l'état, suppression des stacks	✓✓ Réalisé
CU2.2	Architecture complexe multi-VM / multi-réseaux	⚡ Partiel (flat)
CU2.4	Mise à jour d'une infrastructure déployée sans suppression	✓✓ Réalisé
CU3.1 / 3.2 / 3.3	Politiques de scale UP/DOWN et seuils configurables	✓✓ Réalisé
CU3.4	Intégration monitoring externe	⚡ Partiel
CU4.1 / 4.2 / 4.3	Collecte métriques, visualisation temps réel et historique scaling	✓✓ Réalisé
CU4.4	Alertes email / SMS / Slack sur événements de scaling	✗ Non réalisé

8. Problèmes rencontrés & Solutions apportées

P1 Base de données inaccessible

Cause : Chemin relatif 'database/orchestration.db' — dépend du répertoire de lancement

Solution Path(__file__).parent.parent pour construire un chemin absolu ancré sur config.py

P2 Panneau VM s'ouvre automatiquement

Cause : Classe CSS .hidden non définie globalement — .side-panel restait visible malgré la classe

Solution .hidden { display: none !important } + fermerPanneau() au DOMContentLoaded

P3 Graphiques monitoring vides

Cause : Agent envoie hostname 'heat-vm', interface cherche par UUID Nova 'fafb5da2-...' — aucune correspondance

Solution Résolution hostname→UUID dans receive_metrics() avant stockage et émission WebSocket

P4 Resize bloqué en VERIFY_RESIZE

Cause : Nova exige une confirmation explicite après resize — sans confirmation la VM reste bloquée

Solution Thread de fond _thread_resize() surveille l'état et confirme automatiquement

P5 tenant_network_types = flat

Cause : DevStack configuré en flat network — impossible de créer de nouveaux réseaux tenant via Heat

Solution Templates réécris pour utiliser private-network et public-network existants comme paramètres

P6 Erreur 'Unknown Property description'

Cause : Version Neutron DevStack ne supporte pas le champ description dans les règles de SG

Solution Suppression de tous les champs description dans les rules des OS::Neutron::SecurityGroup

8 — CONCLUSION

Conclusion & Perspectives

La plateforme Heat Orchestration répond à l'ensemble des besoins fonctionnels identifiés dans le cahier des charges, en proposant une orchestration automatisée, un monitoring temps réel et un scaling intelligent des ressources cloud.

Architecture MVT + Service Layer

Séparation claire des responsabilités, code maintenable et extensible

Agent de monitoring universel

Résolution hostname→UUID : fonctionnel dans tout environnement OpenStack

Scaling asynchrone robuste

Thread de fond + confirmation automatique Nova = expérience fluide

Composants Heat réutilisables

3 sous-templates modulaires, injectables dans n'importe quel stack

Stack update sans suppression

Les stacks existantes sont préservées lors des modifications (CU2.4)

Perspectives

- Intégration Prometheus / Grafana
- Alertes email / Slack (CU4.4)
- Scaling multi-métriques combinées
- Migration PostgreSQL en production
- Prédiction de charge par IA/ML